

머신러닝 실습 단계별 함수 정리

데이터 불러오기부터 모델 평가까지

이 자료는 머신러닝 실습 중

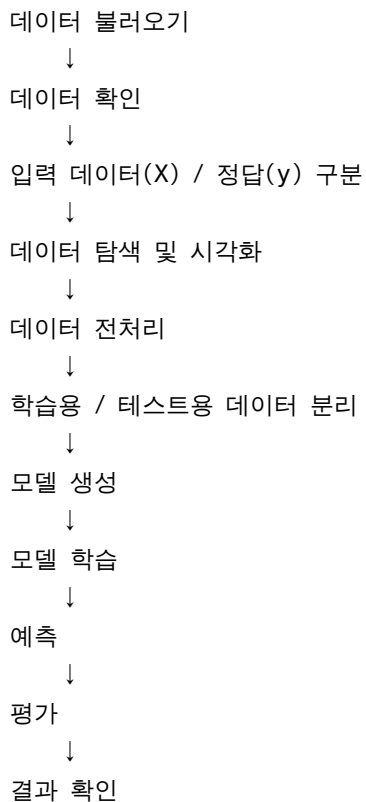
“지금 단계에서 어떤 함수를 써야 하지?”

라는 질문이 생겼을 때 찾아보기 위한 참고자료이다.

모든 함수와 옵션을 외울 필요는 없다.

전체 흐름을 먼저 이해하고, 필요한 함수는 찾아서 사용한다.

1. 머신러닝 기본 흐름



문제에 따라 일부 단계는 생략되거나 순서가 달라질 수 있다.

2. 데이터 불러오기

CSV 파일 불러오기

```
import pandas as pd

df = pd.read_csv('파일명.csv')
```

`pd.read_csv()`

항목	내용
기능	CSV 파일을 읽어 DataFrame으로 변환
주요 입력	파일 경로
결과	DataFrame

자주 사용하는 옵션:

```
pd.read_csv('data.csv', encoding='utf-8')
pd.read_csv('data.csv', index_col=0)
```

3. 데이터 일부 확인하기

`head()`

```
df.head()
```

앞의 5개 행을 확인한다.

```
df.head(10)
```

앞의 10개 행을 확인한다.

`tail()`

```
df.tail()
```

마지막 5개 행을 확인한다.

4. 데이터 크기 확인하기

`.shape`

```
df.shape
```

예:

```
(1000, 8)
```

→ 행 1000개

→ 열 8개

5. 데이터 기본 정보 확인하기

`info()`

```
df.info()
```

확인할 수 있는 것:

- 행 개수
 - 열 이름
 - 각 열의 자료형
 - 결측치 여부
 - 메모리 사용량
-

`dtypes`

```
df.dtypes
```

각 열의 자료형을 확인한다.

예:

```
temperature    float64
pressure       float64
```

machine	object
failure	int64

6. 수치 데이터 요약하기

`describe()`

```
df.describe()
```

주로 다음 값을 확인한다.

- count
- mean
- std
- min
- 25%
- 50%
- 75%
- max

특정 열:

```
df['temperature'].describe()
```

7. 열 이름 확인하기

```
df.columns
```

리스트 형태가 필요하면:

```
df.columns.tolist()
```

8. 특정 열 선택하기

한 개 열

```
df['temperature']
```

결과:

```
Series
```

여러 열

```
df[['temperature', 'pressure']]
```

결과:

```
DataFrame
```

머신러닝 입력 데이터 ☒는 일반적으로 여러 열로 구성된 2차원 형태를 사용한다.

9. 특정 행 선택하기

`iloc`

행 번호를 기준으로 선택한다.

```
df.iloc[0]
```

첫 번째 행

```
df.iloc[0:5]
```

0~4번째 행

loc

인덱스 이름이나 조건을 기준으로 선택한다.

```
df.loc[0]
```

조건 사용:

```
df.loc[df['temperature'] > 100]
```

10. 조건에 맞는 데이터 선택하기

```
df[df['temperature'] > 100]
```

여러 조건:

```
df[
    (df['temperature'] > 100) &
    (df['pressure'] > 50)
]
```

조건 연산자

연산자	의미
>	초과
<	미만
>=	이상
<=	이하
==	같다
!=	같지 않다
&	그리고
\	또는

11. 값의 종류 확인하기

`unique()`

```
df['failure'].unique()
```

해당 열에 어떤 값들이 존재하는지 확인한다.

`nunique()`

```
df['failure'].nunique()
```

서로 다른 값의 개수를 확인한다.

12. 값별 데이터 개수 확인하기

`value_counts()`

```
df['failure'].value_counts()
```

예:

```
0    950
1     50
```

분류 문제에서 클래스 불균형을 확인할 때 자주 사용한다.

비율:

```
df['failure'].value_counts(normalize=True)
```

13. 그룹별 데이터 확인하기

`groupby()`

```
df.groupby('machine')['temperature'].mean()
```

의미:

machine별로 그룹을 나누고
→ temperature의 평균 계산

여러 열:

```
df.groupby('machine')[['temperature', 'pressure']].mean()
```

14. 결측치 확인하기

`isnull()`

```
df.isnull()
```

각 데이터가 결측치인지 `True / False`로 반환한다.

열별 결측치 개수

```
df.isnull().sum()
```

실습에서는 이 형태를 가장 자주 사용한다.

15. 결측치 제거하기

`dropna()`

```
df.dropna()
```


결측치가 포함된 행을 제거한다.

원본에 저장하려면:

```
df = df.dropna()
```

16. 결측치 채우기

`fillna()`

특정 값으로 채우기:

```
df['temperature'] = df['temperature'].fillna(0)
```

평균으로 채우기:

```
mean_value = df['temperature'].mean()

df['temperature'] = (
    df['temperature'].fillna(mean_value)
)
```

17. 중복 데이터 확인하기

```
df.duplicated()
```

중복 행 개수:

```
df.duplicated().sum()
```

중복 제거:

```
df = df.drop_duplicates()
```

18. 데이터 정렬하기

`sort_values()`

```
df.sort_values('temperature')
```

내림차순:

```
df.sort_values(  
    'temperature',  
    ascending=False  
)
```

19. 새로운 열 만들기

```
df['difference'] = (  
    df['temperature'] - df['reference']  
)
```

기존 데이터를 이용해 새로운 특성을 만들 때 사용한다.

20. 데이터 삭제하기

`drop()`

열 삭제:

```
df = df.drop(columns=['id'])
```

여러 열:

```
df = df.drop(  
    columns=['id', 'timestamp']  
)
```

21. 입력 데이터 X와 정답 y 나누기

머신러닝에서 일반적으로:

```
X = 모델에게 입력할 데이터  
y = 모델이 맞혀야 하는 정답
```

예:

```
X = df[  
    ['temperature', 'pressure', 'speed']  
]  
  
y = df['failure']
```

22. 데이터 분포 확인하기

히스토그램

```
import matplotlib.pyplot as plt  
  
plt.hist(df['temperature'])  
plt.show()
```

구간 개수 지정:

```
plt.hist(  
    df['temperature'],  
    bins=20  
)  
  
plt.show()
```

23. 두 변수의 관계 확인하기

산점도

```
plt.scatter(  
    df['temperature'],  
    df['pressure']  
)  
  
plt.show()
```

축 이름:

```
plt.xlabel('temperature')  
plt.ylabel('pressure')  
plt.show()
```

24. 범주별 데이터 개수 시각화

Pandas를 이용한 간단한 방법:

```
df['failure'].value_counts().plot(  
    kind='bar'  
)  
  
plt.show()
```

25. 상관계수 계산하기

`corr()`

```
df.corr(numeric_only=True)
```

특정 변수와의 상관계수:

```
df.corr(  
    numeric_only=True  
)['failure']
```

26. 상관관계 히트맵

Seaborn을 사용하는 경우:

```
import seaborn as sns  
  
corr = df.corr(numeric_only=True)  
  
sns.heatmap(  
    corr,  
    annot=True,  
    cmap='coolwarm'  
)  
  
plt.show()
```

주요 옵션

옵션	기능
<code>annot=True</code>	숫자 표시
<code>fmt=' .2f '</code>	소수 둘째 자리까지 표시
<code>cmap=</code>	색상 표현 방식

27. 범주형 데이터를 숫자로 변환하기

원핫 인코딩

```
pd.get_dummies(df)
```

특정 열만 변환:

```
df = pd.get_dummies(  
    df,
```

```
columns=['machine']
)
```

0, 1 형태로 변환:

```
df = pd.get_dummies(
    df,
    columns=['machine'],
    dtype=int
)
```

28. 학습 데이터와 테스트 데이터 나누기

```
from sklearn.model_selection import train_test_split
```

기본 사용:

```
X_train, X_test, y_train, y_test = (
    train_test_split(
        X,
        y,
        test_size=0.2,
        random_state=42
    )
)
```

주요 파라미터

파라미터	의미
X	입력 데이터
y	정답
test_size	테스트 데이터 비율
random_state	데이터 분할 재현을 위한 값
shuffle	데이터를 섞을지 여부
stratify	클래스 비율을 유지하며 분할

분류 문제:

```
X_train, X_test, y_train, y_test = (
    train_test_split(
        X,
        y,
        test_size=0.2,
        random_state=42,
        stratify=y
    )
)
```

29. 데이터 크기 확인하기

데이터를 나눈 후 확인하는 습관을 들이면 좋다.

```
print(X_train.shape)
print(X_test.shape)

print(y_train.shape)
print(y_test.shape)
```

30. 데이터 표준화하기

```
from sklearn.preprocessing import StandardScaler
```

모델 생성:

```
scaler = StandardScaler()
```

학습 데이터로 기준 계산:

```
scaler.fit(X_train)
```

변환:

```
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

한 번에:

```
X_train_scaled = (  
    scaler.fit_transform(X_train)  
)  
  
X_test_scaled = (  
    scaler.transform(X_test)  
)
```

중요

테스트 데이터에는 일반적으로:

```
fit_transform()
```

이 아니라

```
transform()
```

을 사용한다.

31. 선형 회귀 모델 만들기

```
from sklearn.linear_model import LinearRegression  
  
model = LinearRegression()
```

32. 로지스틱 회귀 모델 만들기

```
from sklearn.linear_model import LogisticRegression  
  
model = LogisticRegression()
```

반복 횟수 조정:

```
model = LogisticRegression(  
    max_iter=1000  
)
```


33. 의사결정나무 모델 만들기

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(
    random_state=42
)
```

34. 랜덤 포레스트 모델 만들기

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    random_state=42
)
```

35. 모델 학습하기

`fit()`

```
model.fit(
    X_train,
    y_train
)
```

전처리된 데이터를 사용한다면:

```
model.fit(
    X_train_scaled,
    y_train
)
```

기억하기

```
fit = 학습
```

모델이 `X_train` 과 `y_train` 의 관계를 학습한다.

36. 모델로 예측하기

`predict()`

```
y_pred = model.predict(X_test)
```

전처리된 데이터:

```
y_pred = model.predict(  
    X_test_scaled  
)
```

기억하기

```
predict = 예측
```

37. 새로운 데이터 하나 예측하기

예:

```
new_data = [[100, 50, 1500]]  
  
model.predict(new_data)
```

단, 모델 학습 전에 스케일링을 했다면 새로운 데이터도 동일하게 변환해야 한다.

```
new_scaled = scaler.transform(  
    new_data  
)  
  
model.predict(new_scaled)
```

38. 분류 확률 확인하기

```
predict_proba()
```

```
model.predict_proba(X_test)
```

예:

```
[[0.92 0.08]  
 [0.23 0.77]]
```

각 행은 각 클래스에 속할 확률을 의미한다.

39. 모델의 학습 결과 확인하기

선형회귀 / 로지스틱회귀 등:

```
model.coef_
```

절편:

```
model.intercept_
```

주의

`coef_`, `intercept_`는 함수가 아니라 속성이다.

따라서 괄호를 사용하지 않는다.

```
model.coef_
```

O

```
model.coef_()
```

X

40. 회귀 모델 평가하기

MAE

```
from sklearn.metrics import mean_absolute_error

mae = mean_absolute_error(
    y_test,
    y_pred
)
```

의미:

실제값과 예측값의 차이의 절댓값 평균

작을수록 좋다.

41. MSE

```
from sklearn.metrics import mean_squared_error

mse = mean_squared_error(
    y_test,
    y_pred
)
```

오차를 제곱한 뒤 평균을 계산한다.

작을수록 좋다.

42. RMSE

```
import numpy as np

rmse = np.sqrt(
    mean_squared_error(
        y_test,
        y_pred
    )
)
```

```
)  
)
```

MSE에 제곱근을 적용한 값이다.

43. 결정계수 R^2

```
from sklearn.metrics import r2_score  
  
r2 = r2_score(  
    y_test,  
    y_pred  
)
```

대략적으로:

1에 가까울수록
모델이 데이터의 변화를 잘 설명함

모델 자체에서도:

```
model.score(  
    X_test,  
    y_test  
)
```

를 사용할 수 있지만, `score()`의 의미는 모델 종류에 따라 달라질 수 있다.

44. 실제값과 예측값 비교하기

```
result = pd.DataFrame({  
    'Actual': y_test,  
    'Predicted': y_pred  
})  
  
result.head()
```

회귀 모델을 이해할 때 매우 유용하다.

45. 실제값과 예측값 시각화하기

```
plt.scatter(  
    y_test,  
    y_pred  
)  
  
plt.xlabel('Actual')  
plt.ylabel('Predicted')  
  
plt.show()
```

예측이 정확할수록 실제값과 예측값이 비슷해진다.

46. 분류 정확도 계산하기

`accuracy_score()`

```
from sklearn.metrics import accuracy_score  
  
accuracy = accuracy_score(  
    y_test,  
    y_pred  
)
```

예:

0.9

→ 전체 데이터의 90%를 올바르게 분류

47. 혼동행렬 확인하기

`confusion_matrix()`

```
from sklearn.metrics import confusion_matrix  
  
cm = confusion_matrix(  
    y_test,  
    y_pred
```

```
)  
  
print(cm)
```

이진분류 예:

```
[[45  5]  
 [ 3 47]]
```

실제 클래스와 예측 클래스의 조합별 개수를 확인한다.

48. 분류 평가 지표 한 번에 보기

`classification_report()`

```
from sklearn.metrics import classification_report  
  
print(  
    classification_report(  
        y_test,  
        y_pred  
    )  
)
```

주요 결과:

- precision
- recall
- f1-score
- support

49. Precision

```
from sklearn.metrics import precision_score  
  
precision_score(  
    y_test,  
    y_pred  
)
```

모델이 Positive라고 예측한 것 중 실제 Positive의 비율.

50. Recall

```
from sklearn.metrics import recall_score

recall_score(
    y_test,
    y_pred
)
```

실제 Positive 중 모델이 찾아낸 비율.

고장 탐지처럼 놓치면 문제가 되는 상황에서 중요할 수 있다.

51. F1-score

```
from sklearn.metrics import f1_score

f1_score(
    y_test,
    y_pred
)
```

Precision과 Recall을 함께 고려한 평가 지표.

52. 학습 데이터 성능과 테스트 데이터 성능 비교하기

```
train_score = model.score(
    X_train,
    y_train
)

test_score = model.score(
    X_test,
    y_test
)

print(train_score)
print(test_score)
```

두 값의 차이가 매우 크다면 과적합 가능성을 살펴볼 수 있다.

53. 모델의 주요 설정 확인하기

```
model.get_params()
```

예:

```
model.get_params()
```

모델을 만들 때 설정한 파라미터를 확인할 수 있다.

54. 자주 사용하는 import 모음

데이터 처리

```
import numpy as np
import pandas as pd
```

시각화

```
import matplotlib.pyplot as plt
import seaborn as sns
```

데이터 분할

```
from sklearn.model_selection import train_test_split
```

전처리

```
from sklearn.preprocessing import StandardScaler
```

회귀

```
from sklearn.linear_model import LinearRegression
```

분류

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
```

회귀 평가

```
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)
```

분류 평가

```
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    precision_score,
    recall_score,
    f1_score
)
```

55. 회귀 실습 기본 뼈대

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

```

# 1. 데이터 불러오기
df = pd.read_csv('data.csv')

# 2. X, y 설정
X = df[['feature1', 'feature2']]
y = df['target']

# 3. 데이터 분할
X_train, X_test, y_train, y_test = (
    train_test_split(
        X,
        y,
        test_size=0.2,
        random_state=42
    )
)

# 4. 모델 생성
model = LinearRegression()

# 5. 학습
model.fit(X_train, y_train)

# 6. 예측
y_pred = model.predict(X_test)

# 7. 평가
mse = mean_squared_error(
    y_test,
    y_pred
)

r2 = r2_score(
    y_test,
    y_pred
)

print('MSE:', mse)
print('R2:', r2)

```

56. 분류 실습 기본 뼈대

```
import pandas as pd
```

```

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix
)

# 1. X, y 설정
X = df[['feature1', 'feature2']]
y = df['target']

# 2. 데이터 분할
X_train, X_test, y_train, y_test = (
    train_test_split(
        X,
        y,
        test_size=0.2,
        random_state=42,
        stratify=y
    )
)

# 3. 모델 생성
model = LogisticRegression(
    max_iter=1000
)

# 4. 학습
model.fit(X_train, y_train)

# 5. 예측
y_pred = model.predict(X_test)

# 6. 평가
accuracy = accuracy_score(
    y_test,
    y_pred
)

cm = confusion_matrix(
    y_test,
    y_pred
)

print('Accuracy:', accuracy)
print(cm)

```

57. 표준화가 포함된 분류 기본 뼈대

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

X_train, X_test, y_train, y_test = (
    train_test_split(
        X,
        y,
        test_size=0.2,
        random_state=42,
        stratify=y
    )
)

scaler = StandardScaler()

X_train_scaled = (
    scaler.fit_transform(X_train)
)

X_test_scaled = (
    scaler.transform(X_test)
)

model = LogisticRegression()

model.fit(
    X_train_scaled,
    y_train
)

y_pred = model.predict(
    X_test_scaled
)

accuracy = accuracy_score(
    y_test,
    y_pred
)

print(accuracy)
```

58. 헛갈릴 때 보는 초압축판

데이터 읽기

```
pd.read_csv()
```

데이터 보기

```
head()
```

```
info()
```

```
describe()
```

```
shape
```

값 확인

```
unique()
```

```
value_counts()
```

그룹 분석

```
groupby()
```

결측치

```
isnull()
```

```
dropna()
```

```
fillna()
```

열 선택

```
df['col']
```

```
df[['col1', 'col2']]
```

원핫 인코딩

```
pd.get_dummies()
```

입력/정답

```
X = ...
```

```
y = ...
```

데이터 나누기

```
train_test_split()
```

표준화

```
StandardScaler()
```

```
fit_transform()
```

```
transform()
```

모델 만들기

```
LinearRegression()
```

```
LogisticRegression()
```

학습

`fit()`

예측

`predict()`

확률

`predict_proba()`

회귀 평가

`mean_absolute_error()`

`mean_squared_error()`

`r2_score()`

분류 평가

`accuracy_score()`

`confusion_matrix()`

`classification_report()`

학습 결과

`coef_`

`intercept_`

59. 함수 이름보다 더 중요한 것

머신러닝 코드를 외우기보다 다음 흐름을 기억한다.

- ① 데이터를 준비한다.
- ② 무엇을 입력(X)으로 사용하고
무엇을 예측(y)할지 정한다.
- ③ 학습 데이터와 테스트 데이터를 나눈다.
- ④ 필요하면 전처리한다.
- ⑤ 모델을 만든다.
- ⑥ `fit()`으로 학습한다.
- ⑦ `predict()`로 예측한다.
- ⑧ 실제값과 비교하여 평가한다.

함수 이름이 기억나지 않는 것은 큰 문제가 아니다.

지금 머신러닝 과정의 어느 단계에 있는지를 아는 것이 더 중요하다.